Lab 1 Report

Lab Assurance, HTTP Mapping and Attack-Surface Reconnaissance

---

Student Name: Daniyal Ahmed
Registration Number: C11/26/EHIT/17331
Cohort: Ethical Hacking Internship Training
Course: Offensive Security Operations I (CIP-A104)
Submission Date: 26 August 2026
Assessment: Lab 1

Academic Integrity Statement

I confirm that all requests, responses, screenshots, tool output, observations and conclusions in this report are my own. No walkthrough was copied without reproducing the evidence personally. No out-of-scope or publicly accessible system was tested at any point.

1. Environment Assurance & VM Topology

The lab was conducted entirely within an isolated VMware Workstation virtual machine running Kali Linux. Two network adapters were configured to provide layered isolation:

Parameter	Value
Hypervisor	VMware Workstation
Guest OS	Kali Linux
Network Adapter 1	NAT — VMware-internal subnet 192.168.110.0/24, not routable to physical LAN or internet
Network Adapter 2	Host-only — no IP assigned, fully isolated from all external networks
VM IP (eth0)	192.168.110.141/24
DVWA URL	http://127.0.0.1:42001/
Mutillidae URL	http://127.0.0.1:42002/

1.1 Isolation Mechanism

Both applications use a standalone nginx instance with an explicit source-IP access control list. The configuration block in /etc/dvwa/vhost/dvwa-nginx.conf and the equivalent Mutillidae vhost enforce:

```
allow 127.0.0.1;
deny all;
```

This means nginx rejects any HTTP request whose source IP is not 127.0.0.1 (localhost), regardless of which network interface the socket is listening on. This provides a second layer of isolation on top of the NAT network boundary — a defense-in-depth model where compromise of one control (e.g., a bridged adapter accidentally enabled) would still be blocked by the application-layer ACL.

1.2 Isolation Evidence

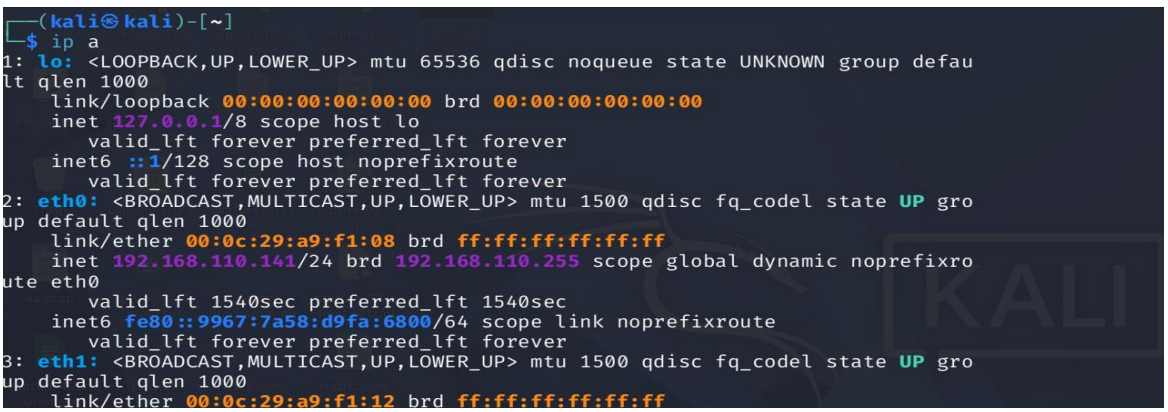


Figure: Kali VM ip a output — eth0 (192.168.110.141/24, NAT), eth1 (Host-only, no IP)

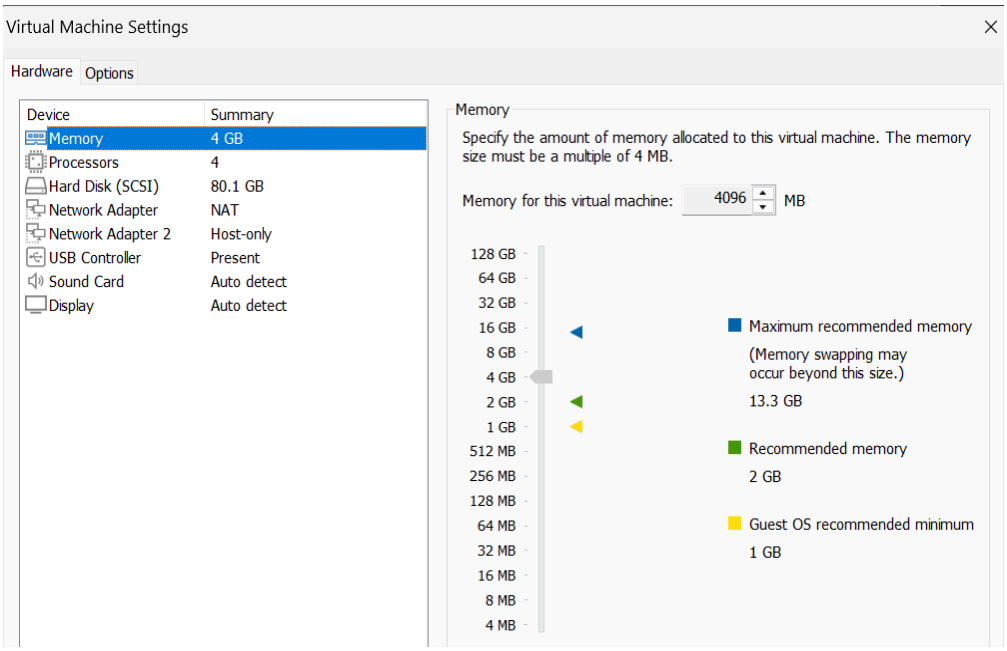


Figure: VMware VM Settings — Network Adapter 1: NAT, Network Adapter 2: Host-only

Isolation cross-check from Windows host machine:

```
curl -s -o /dev/null -w "%{http_code}\n" http://192.168.110.141:42001/
```

Result: HTTP 403 — nginx's allow/deny ACL rejected the non-loopback source IP. This confirms the application is reachable from the host (expected for NAT) but any request not originating from 127.0.0.1 is rejected before reaching DVWA's PHP code. The same test against port 42002 (Mutillidae) returned HTTP 403 identically.

```
C:\Users\Bytes>curl -s -o /dev/null -w "%{http_code}\n" http://192.168.110.141:42001/
403
```

2. Application Readiness

2.1 DVWA — Setup, Reset and Security Level

DVWA was accessed at http://127.0.0.1:42001/ from within the Kali VM. Login was performed using the default credentials (admin / password). The Setup / Reset DB page was visited to confirm system readiness and perform a clean database reset.

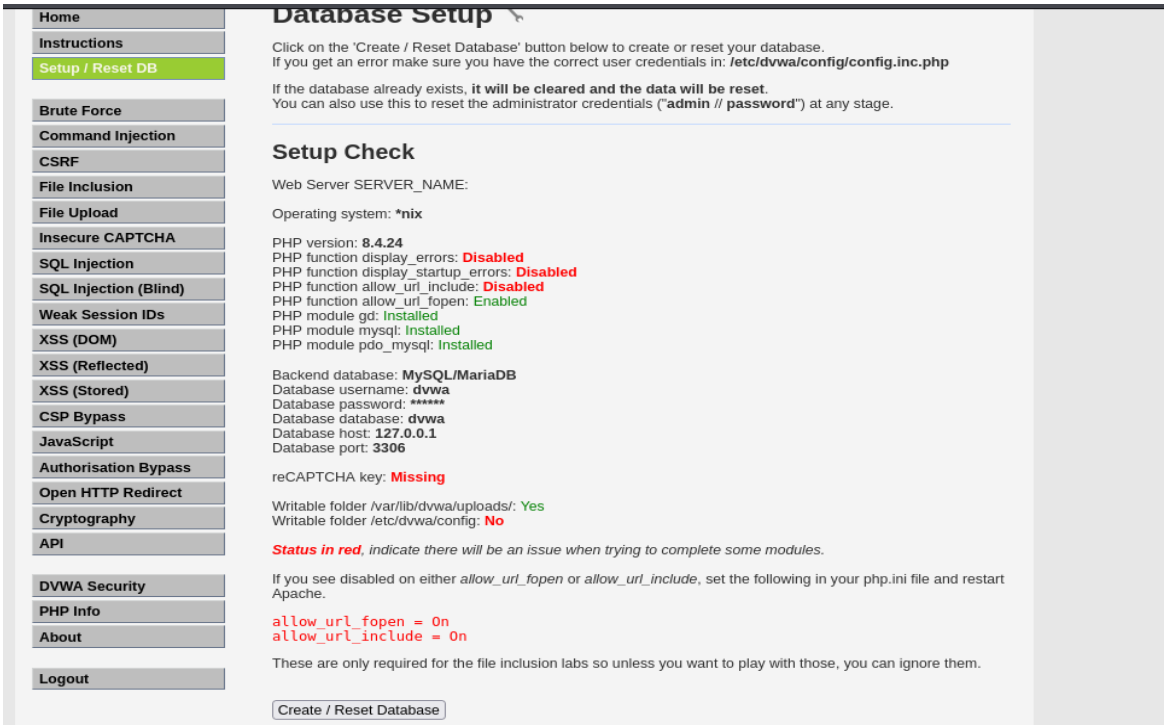


Figure: DVWA Setup Check page — PHP 8.2.4, MySQL/MariaDB backend, key module status

Key values recorded from Setup Check:

Item	Value
Operating System	*nix (Linux)
PHP Version	8.2.4
PHP allow_url_include	Disabled (relevant for remote file inclusion testing)
PHP allow_url_fopen	Enabled
PHP modules	gd, mysql, pdo_mysql — all Installed
Backend database	MySQL / MariaDB
DB Host	127.0.0.1 : 3306
DB Name / User	dvwa / dvwa
Upload directory writable	Yes (/var/lib/dvwa/uploads/)

reCAPTCHA key	Missing (non-blocking for this lab)

The "Create / Reset Database" button was clicked, rebuilding the DVWA schema cleanly. Security level was then set to Low for baseline testing:

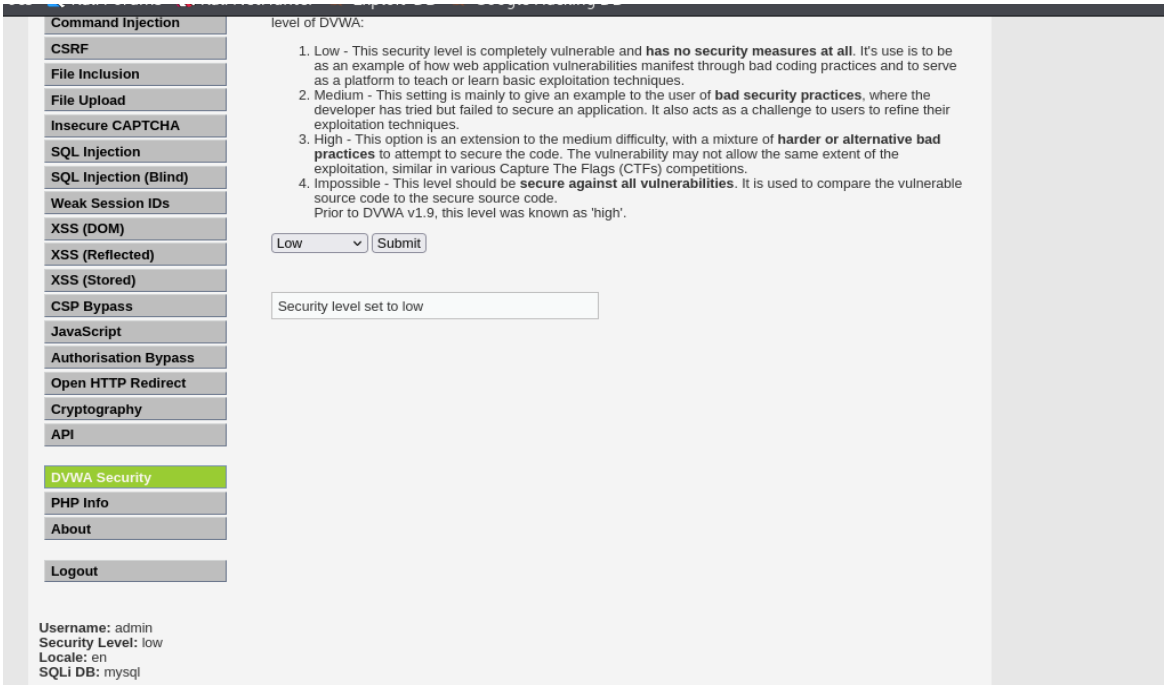


Figure: DVWA Security page — level set to Low, confirmed via footer (Username: admin, Security Level: low, SQLi DB: mysql)

2.2 Mutillidae — Installation, Setup and Reset

Mutillidae was not available as a Kali apt package. It was installed by cloning the official repository to /usr/share/mutillidae/ and configuring a standalone nginx instance mirroring DVWA's service architecture:

- Cloned: https://github.com/webpwnized/mutillidae.git → /usr/share/mutillidae/
- Dedicated system user _mutillidae created (mirrors _dvwa)
- PHP-FPM pool created: /etc/php/8.4/fpm/pool.d/42002.conf
- nginx config: /etc/mutillidae/nginx.conf + vhost on port 42002 with allow 127.0.0.1/deny all
- Dedicated MySQL user and database created: mutillidae / mutillidae
- Database schema built via the built-in set-up-database.php page

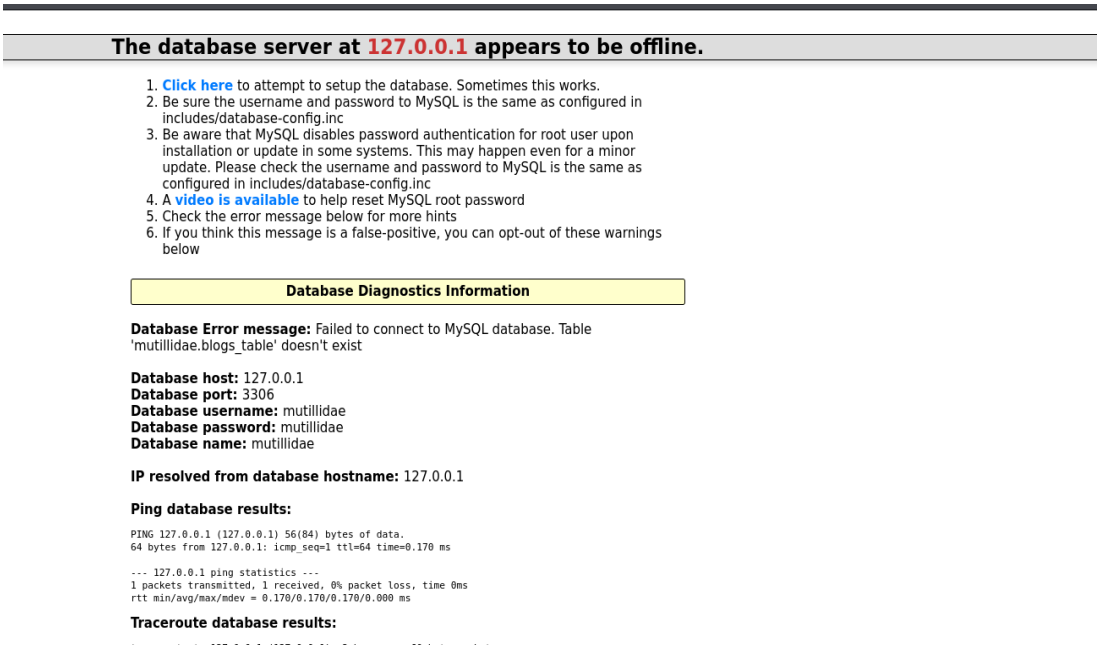
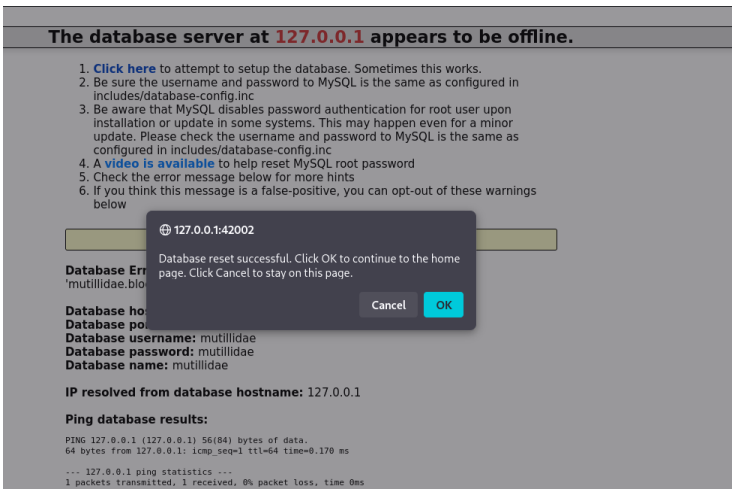


Figure: Mutillidae database connectivity diagnostics — DB credentials correct, schema tables missing before setup



Figure: Mutillidae home page after DB rebuild — Version 2.12.7, Security Level: 0 (Hosed/Insecure), Hints: Enabled

After the DB rebuild, Toggle Security and Toggle Hints were both exercised to confirm these features function correctly. Reset DB was also clicked and confirmed. The application was reverted to its baseline state (insecure/hosed mode, hints enabled).

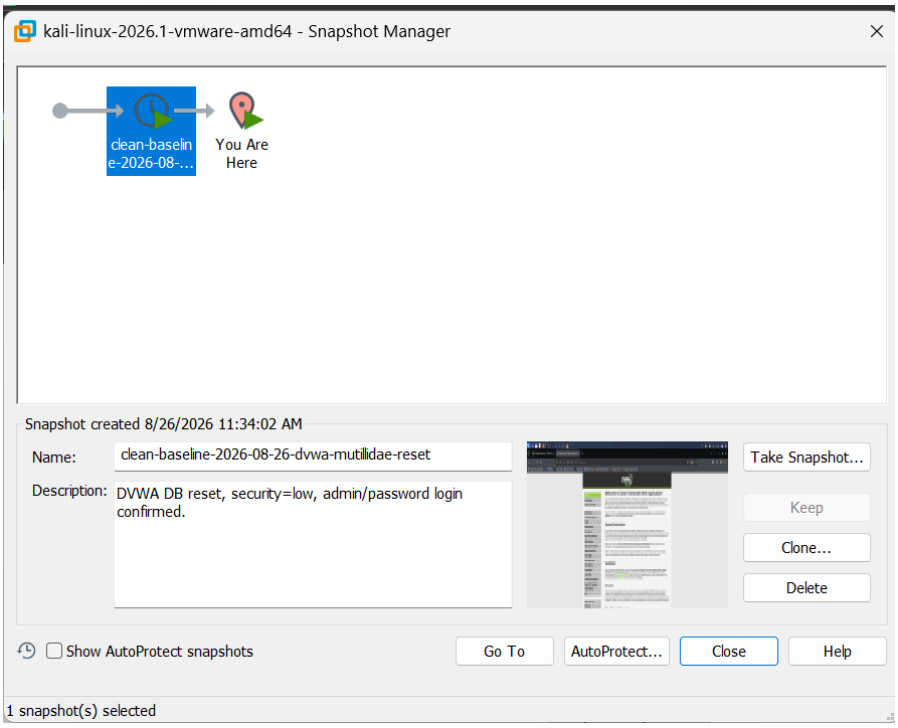


3. Snapshot and Rules of Engagement

3.1 VMware Snapshot

A clean baseline snapshot was taken in VMware after both applications were reset and confirmed working. This snapshot can be used to restore both targets to their tested baseline state at any point during the course window.

Field	Value
Snapshot name	clean-baseline-2026-08-26-dvwa-mutillidae
Description	DVWA DB reset, security=low. Mutillidae DB rebuilt, security=hosed (insecure). Both isolated via nginx allow/deny on 127.0.0.1.
Taken	26 August 2026
State	DVWA login confirmed working, Mutillidae login confirmed working, Burp proxy confirmed capturing traffic from both apps



3.2 Rules of Engagement

Item	Detail
In-scope targets	http://127.0.0.1:42001/ (DVWA) and http://127.0.0.1:42002/ (Mutillidae) — Kali VM only

Excluded	Any host outside this VM, any port other than 42001/42002, the host Windows machine, any real or production system, any internet-facing resource
Testing window	24–26 August 2026 (WAT)
Stop conditions	Any evidence of traffic leaving the NAT/host-only network segment; any accidental bridged-adapter exposure; any credential reuse beyond default app credentials
Authorisation	Individual academic lab assignment — self-authorised against self-owned isolated VM within ICDFA course scope
Out-of-scope statement	No out-of-scope or publicly accessible system was tested at any point during this lab.

4. Proxy Configuration and Baseline Captures

4.1 Burp Suite Setup

Burp Suite Community Edition (v2026.3.2) was used as the interception proxy. A temporary project was created with the default listener on 127.0.0.1:8080. Firefox's proxy was configured via about:preferences and the about:config setting network.proxy.allow_hijacking_localhost was set to true to allow proxying of loopback addresses — this is required because Firefox's native proxy implementation excludes 127.0.0.1 from proxying by default regardless of the "No proxy for" field.

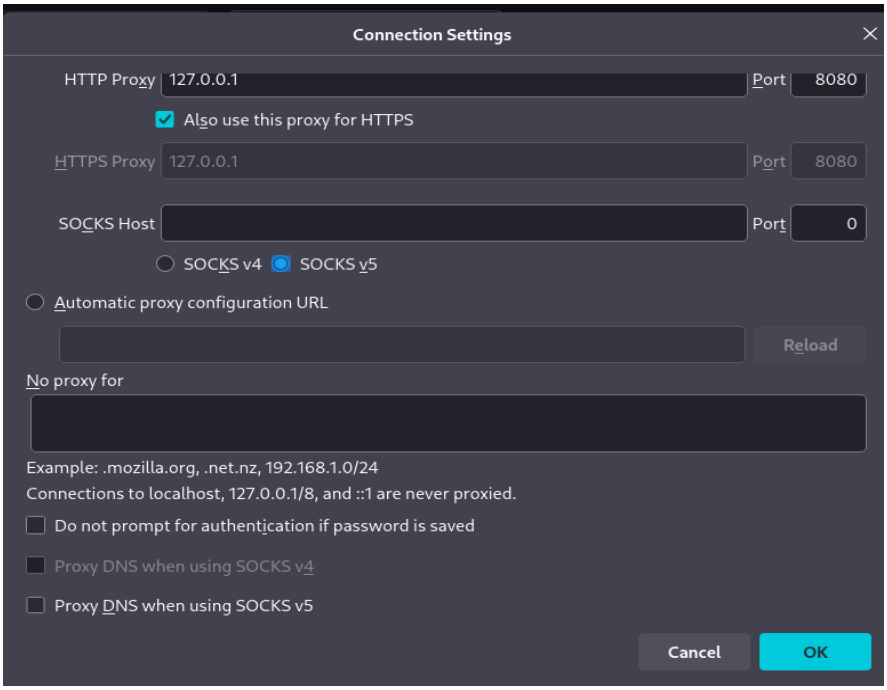


Figure: Firefox Connection Settings — Manual proxy 127.0.0.1:8080, "No proxy for" field empty

4.2 DVWA — GET and POST Captures

The following requests were captured by navigating to the DVWA login page and performing a login with the default credentials.

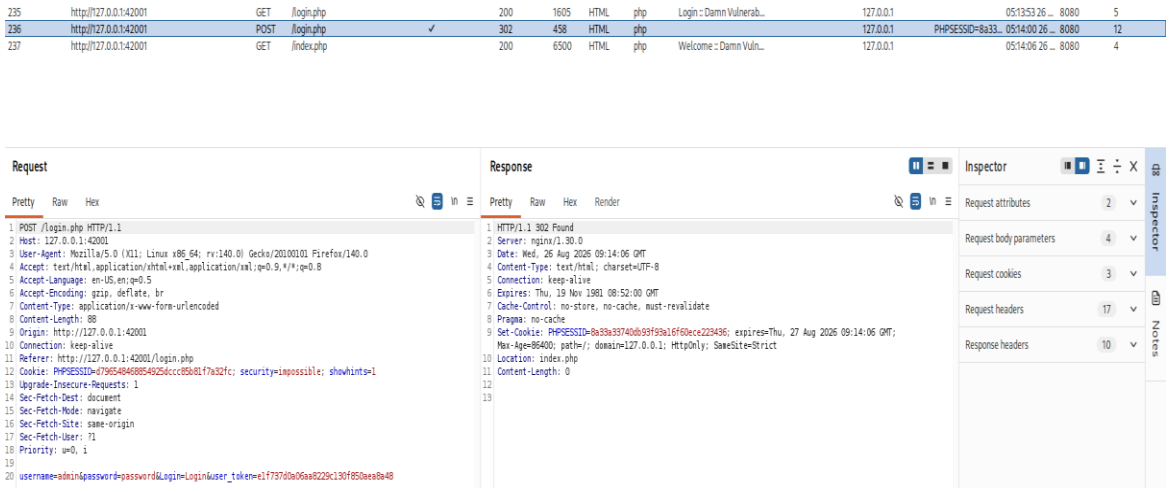


Figure: Burp HTTP History — DVWA GET /login.php (200) and POST /login.php (302) with request/response detail

DVWA GET /login.php

Field	Value
Method	GET
Path	/login.php
Host	127.0.0.1:42001
Status	200 OK
Content-Type	text/html; charset=UTF-8
Response length	1605 bytes
Cookies sent	None (initial request)

DVWA POST /login.php

Field	Value
Method	POST
Path	/login.php
Host	127.0.0.1:42001
Content-Type	application/x-www-form-urlencoded
Body parameters	username=admin, password=password, Login=Login, user_token=e1f737d0a06aa8229c130f850aea8a48
Cookies sent	PHPSESSID=d796548...; security=low; showhints=1
Status	302 Found → Location: index.php

Set-Cookie	PHPSESSID=8a33a337…; HttpOnly; SameSite=Strict; domain=127.0.0.1; Max-Age=86400
Server	nginx/1.30.0

4.3 Mutillidae — GET and POST Captures

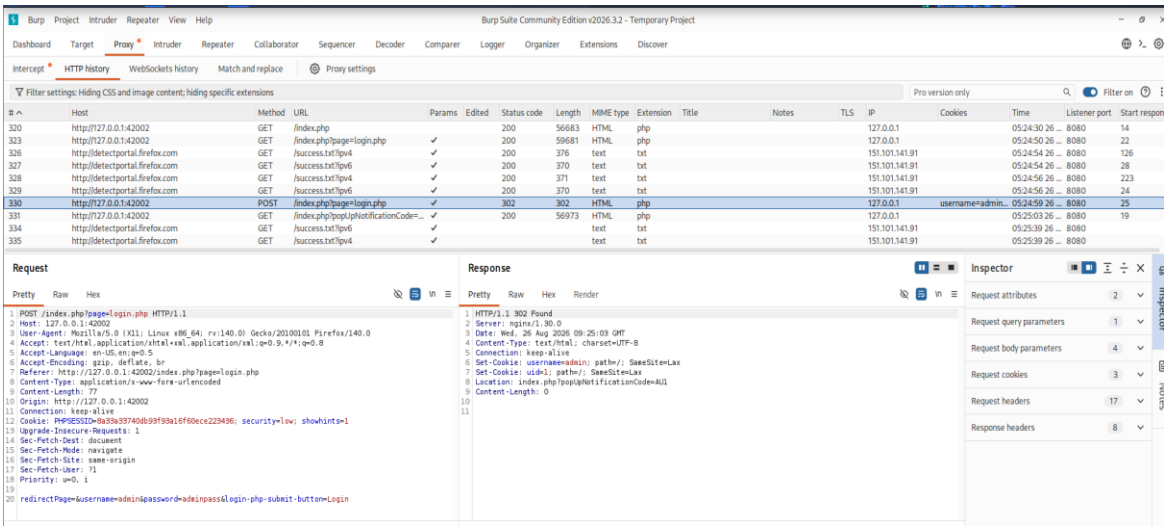


Figure: Burp HTTP History — Mutillidae POST /index.php?page=login.php (302) with full request detail

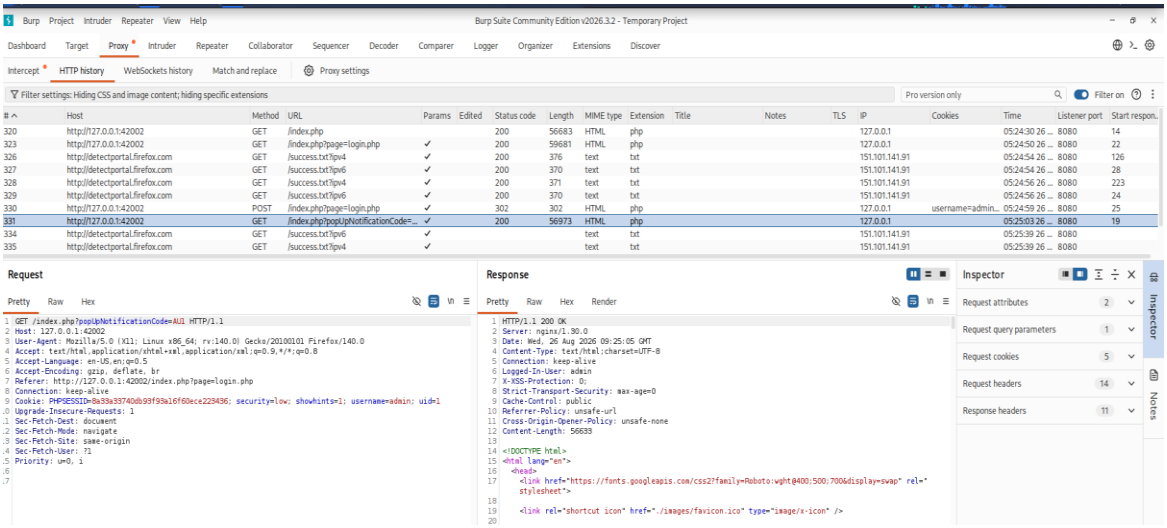


Figure: Burp HTTP History — Mutillidae GET /index.php?popUpNotificationCode=AU1 (200) post-login landing page

Mutillidae GET /index.php?page=login.php

Field	Value
Method	GET
Path	/index.php?page=login.php
Host	127.0.0.1:42002

Status	200 OK
Content-Type	text/html; charset=UTF-8
Response length	59681 bytes

Mutillidae POST /index.php?page=login.php

Field	Value
Method	POST
Path	/index.php?page=login.php
Host	127.0.0.1:42002
Content-Type	application/x-www-form-urlencoded
Body parameters	redirectPage=(empty), username=admin, password=adminpass, login-php-submit-button=Login
Cookies sent	PHPSESSID=8a33a337...; security=low; showhints=1
Status	302 Found → Location: index.php?popUpNotificationCode=AU1
Set-Cookie	username=admin; SameSite=Lax (plain, not HttpOnly)
Set-Cookie (2)	uid=1; SameSite=Lax (plain, not HttpOnly)
Server	nginx/1.30.0
Notable	No CSRF token in request. Identity stored in readable client-side cookies.

5. HTTP Mapping — Cookies and Input Points

5.1 Cookie Inventory

Cookie	Application	Attributes	Purpose
PHPSESSID	DVWA	HttpOnly; SameSite=Strict; rotated on login	PHP session identifier
security	DVWA	Plain (not HttpOnly)	Active security level (low/medium/high/impossible)
showhints	DVWA	Plain	Hint display toggle
PHPSESSID	Mutillidae	Not HttpOnly in observed captures	PHP session identifier
security	Mutillidae	Plain, binary (0/1)	Security mode (0=hosed/insecure, 1=secure)

showhints	Mutillidae	Plain	Hint display toggle
username	Mutillidae	Plain; SameSite=Lax; NOT HttpOnly; set post-login	Client-side identity marker — readable by JS
uid	Mutillidae	Plain; SameSite=Lax; NOT HttpOnly; set post-login	Client-side user ID — readable by JS

Note: Since both applications run on 127.0.0.1 differing only by port, browser cookies are not port-scoped. Mutillidae's username and uid cookies were observed being sent in requests to DVWA — a cross-application cookie leakage artifact of the shared-host lab environment.

5.2 Input Point Inventory — DVWA

Vulnerability Module	Method	Path	Parameters
Login	GET / POST	/login.php	username, password, Login, user_token (CSRF)
SQL Injection	GET	/vulnerabilities/sqli/	id, Submit
SQL Injection (Blind)	GET	/vulnerabilities/sqli_blind/	id, Submit
File Inclusion	GET	/vulnerabilities/fi/	page (file path)
File Upload	POST	/vulnerabilities/upload/	MAX_FILE_SIZE (hidden), uploaded (file), Upload
Command Injection	POST	/vulnerabilities/exec/	ip, Submit
XSS (Reflected)	GET	/vulnerabilities/xss_r/	name, Submit
XSS (Stored)	POST	/vulnerabilities/xss_s/	txtName, mtxMessage, btnSign
Brute Force	GET	/vulnerabilities/brute/	username, password, Login
CSRF	POST	/vulnerabilities/csrf/	password_new, password_conf, Change

DVWA Input Evidence — Burp Captures

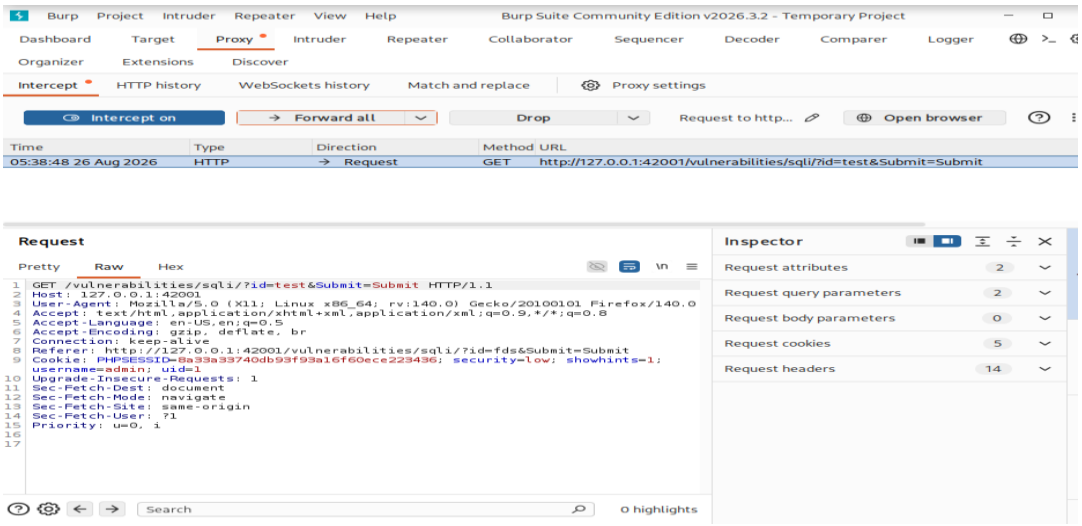


Figure: DVWA SQL Injection — GET /vulnerabilities/sqli/?id=test&Submit=Submit captured in Burp Intercept

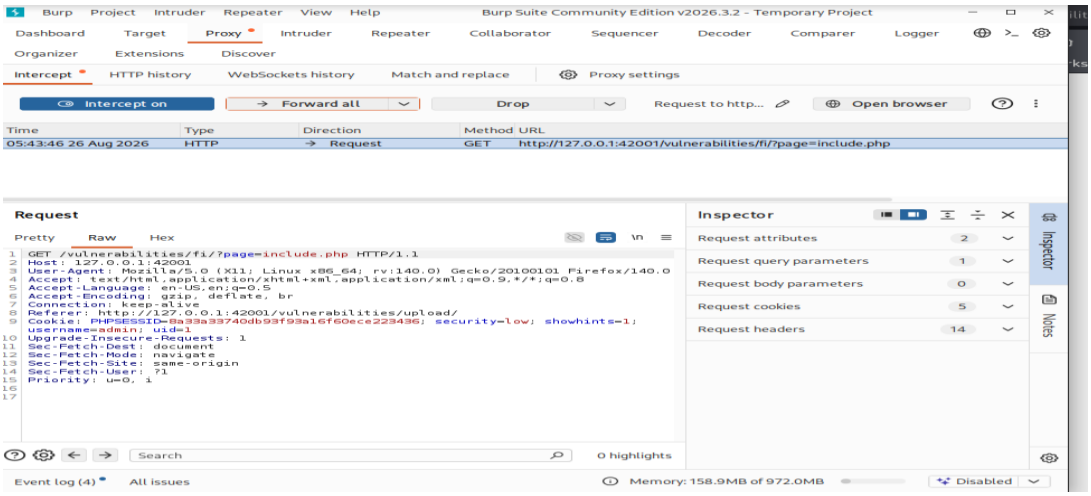


Figure: DVWA File Inclusion — GET /vulnerabilities/fi/?page=include.php captured in Burp Intercept

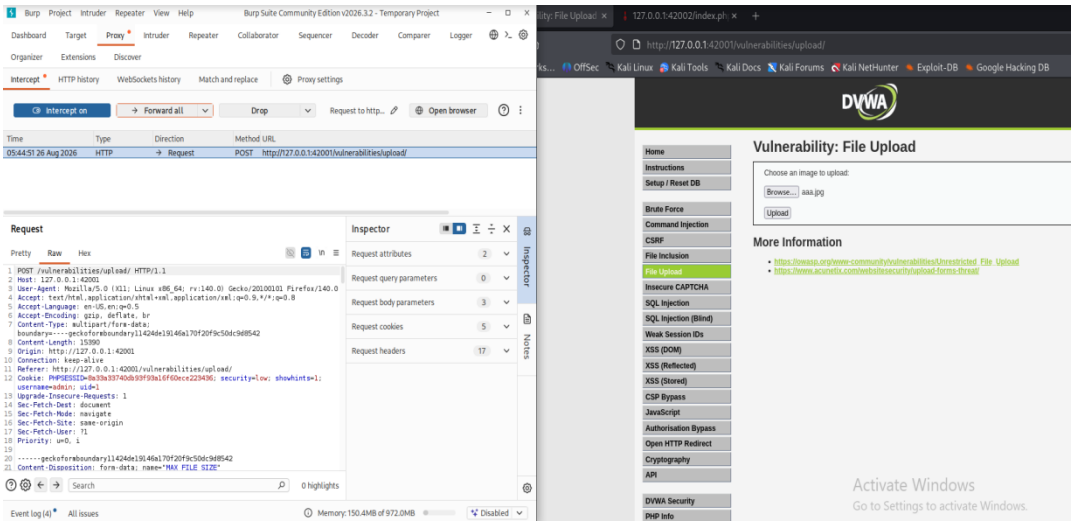
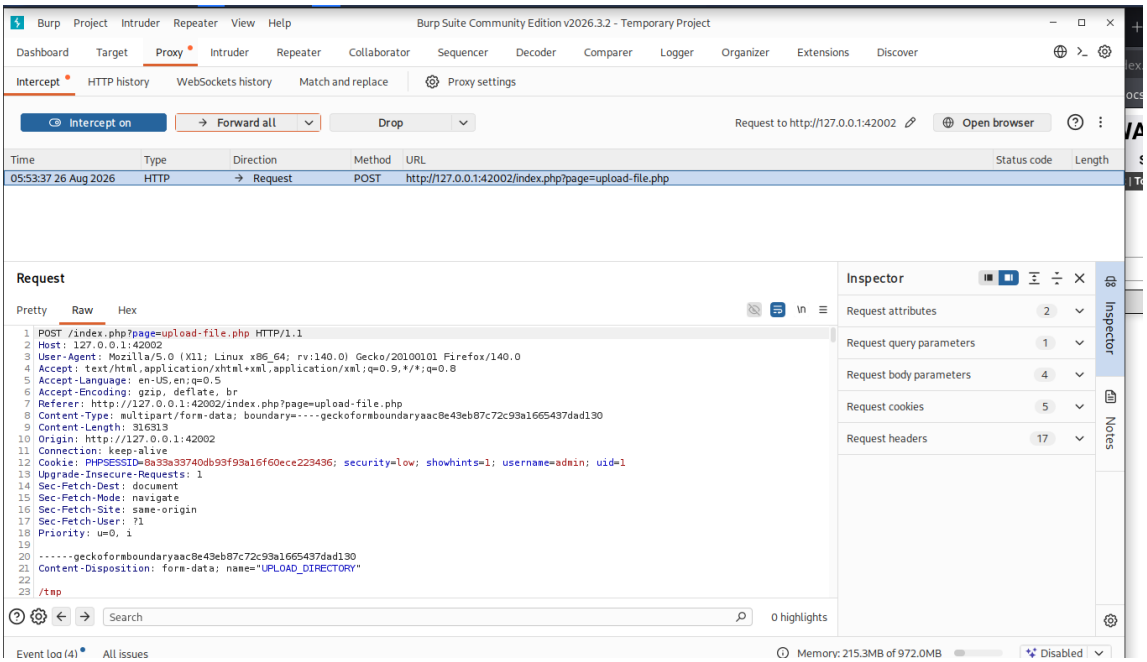
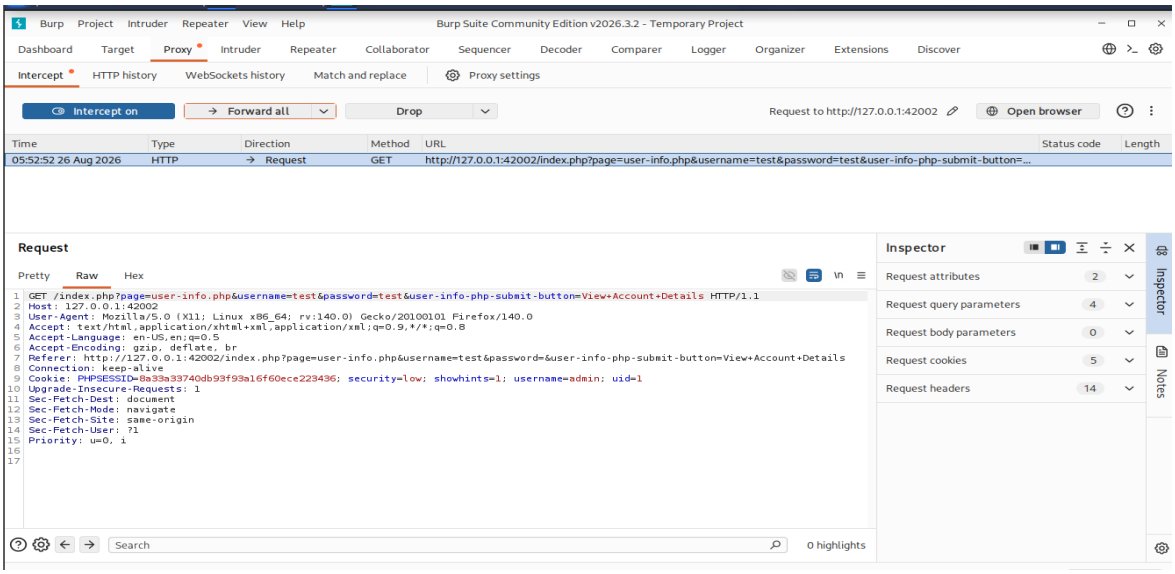


Figure: DVWA File Upload — POST /vulnerabilities/upload/ with multipart/form-data body in Burp Intercept

5.3 Input Point Inventory — Mutillidae

Module	Method	Path / Parameter	Notes
Login	POST	/index.php?page=login.php / username, password	No CSRF token
Register	POST	/register.php / username, password, confirm	No CSRF token
SQL Injection	GET/POST	/user-info.php / username	Multiple SQLi entry points across OWASP categories
Arbitrary File Inclusion	GET	/arbitrary-file-inclusion.php?page=	Direct filename parameter — LFI/RFI surface
File Upload	POST	/upload-file.php / upload-file-form-field	Unrestricted upload, no size check
DNS Lookup	POST	/dns-lookup.php / target_host	OS command injection surface
User Poll	POST	/user-poll.php / choice	Stored XSS surface
Add to Blog	POST	/add-to-your-blog.php / blog_entry	Stored XSS / SQLi
Echo	GET	/echo.php / search_string	Reflected XSS
Text File Viewer	GET	/ext-file-viewer.php / filename	LFI surface





6. Technology Fingerprinting

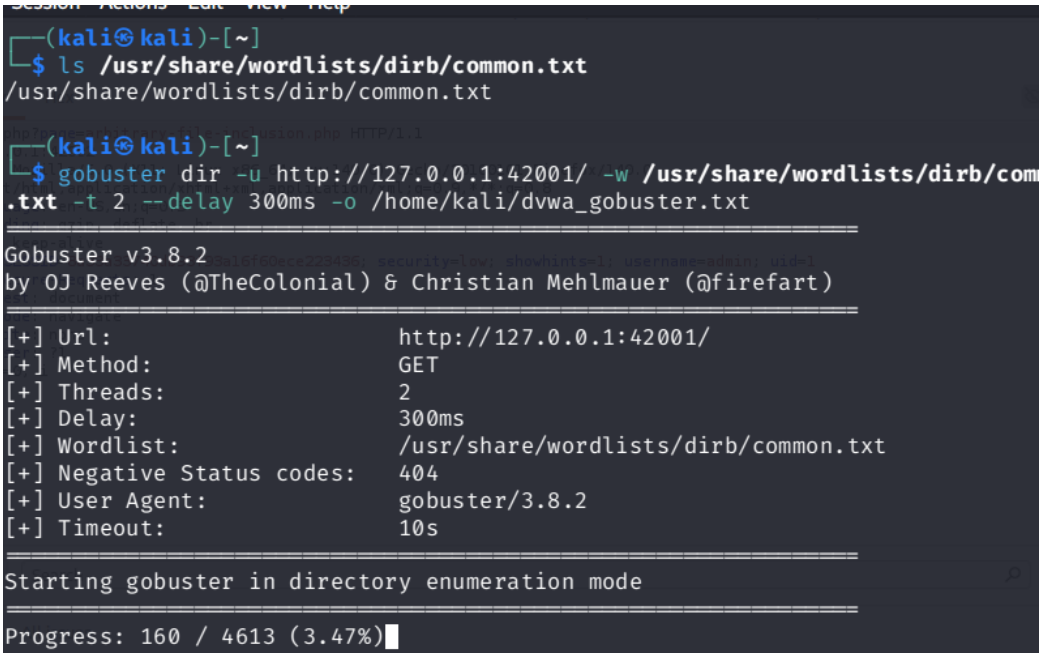
Clue / Source	Value Observed	Likely Technology	Confidence
Server response header	nginx/1.30.0	nginx web server	High
DVWA setup.php	PHP Version: 8.2.4	PHP	High
systemctl	php8.4-fpm.service running	PHP-FPM process manager	High
Cookie name	PHPSESSID	PHP native session handling	High
DVWA setup.php	Backend: MySQL/MariaDB	MariaDB database	High
DVWA setup.php	OS: *nix	Linux operating system	High
Mutillidae response header	Logged-In-User: admin	Mutillidae custom app header	High
Mutillidae response header	X-XSS-Protection: 0	Intentionally disabled XSS filter (training design)	High
Mutillidae response header	Referrer-Policy: unsafe-url	Intentionally permissive referrer policy	High
Mutillidae response header	Cross-Origin-Opener-Policy: unsafe-none	Intentionally permissive COOP	High
Mutillidae response header	Cache-Control: public	No cache protection on authenticated pages	High
Page source	Google Fonts API link	Google Fonts CDN (Mutillidae)	Medium

Mutillidae banner	PHP Curl not installed warning	php-curl extension absent	High
X-Powered-By header	Not observed in captures	PHP version disclosure suppressed	Medium

7. Low-Rate Content Discovery

7.1 Configuration

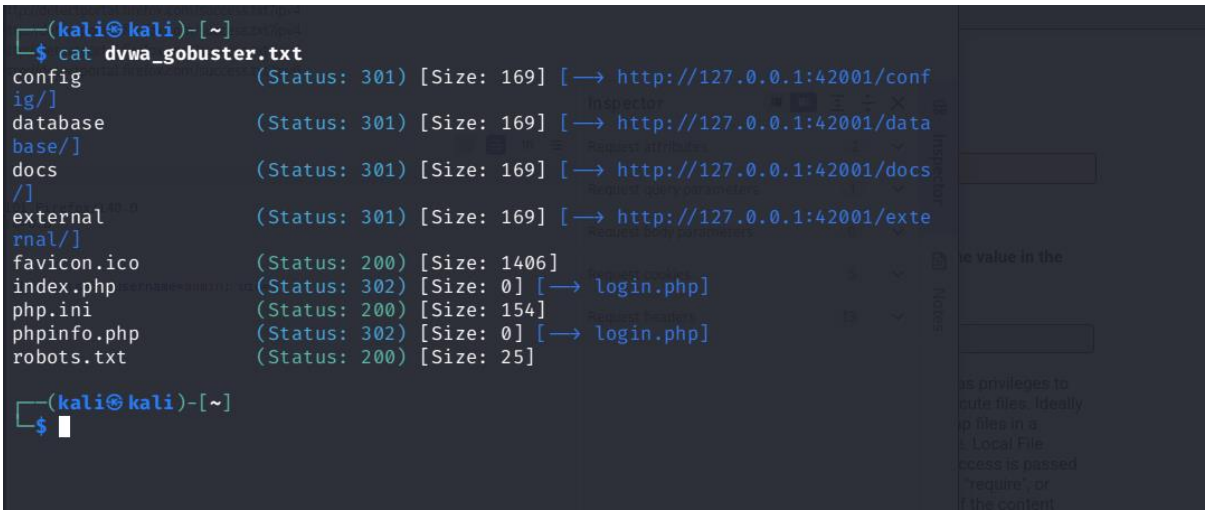
Setting	Value
Tool	gobuster dir
Wordlist	/usr/share/wordlists/dirb/common.txt
Threads	2 (-t 2) — deliberately low concurrency
Request delay	300ms (--delay 300ms) — deliberately low rate
Scope	http://127.0.0.1:42001/ and http://127.0.0.1:42002/ only
Output files	/home/kali/dvwa_gobuster.txt and /home/kali/mutillidae_gobuster.txt



7.2 DVWA Discovery Results

Path	Status	Size	Notes
/config/	301	169	Configuration directory — redirect

/database/	301	169	Database-related directory
/docs/	301	169	Documentation directory
/external/	301	169	External resources directory
/favicon.ico	200	1406	Standard icon file
/index.php	302	0	Redirects to login.php — confirms auth gate
/php.ini	200	154	⚠PHP configuration file directly accessible — manually validated
/phpinfo.php	302	0	Redirects to login.php — protected behind auth
/robots.txt	200	25	Robots exclusion file



7.3 Mutillidae Discovery Results

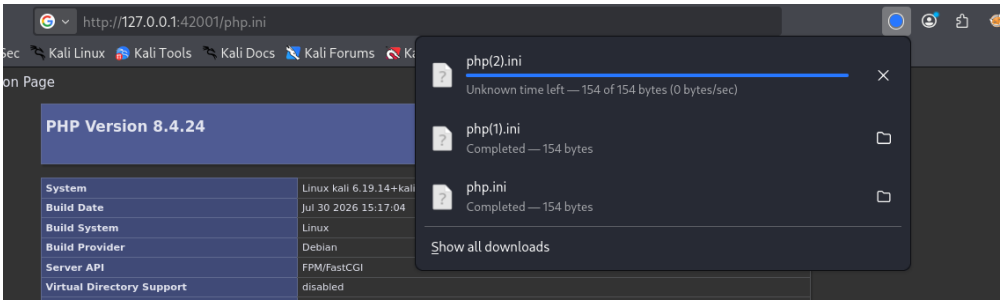
Path	Status	Size	Notes
/.htaccess	200	690	⚠Apache config file directly served — manually validated
/ajax/	301	169	AJAX endpoint directory
/classes/	301	169	PHP class files directory
/data/	301	169	Data directory
/documentation/	301	169	Documentation directory
/images/	301	169	Image assets directory
/includes/	301	169	PHP include files — potentially sensitive

/index.php	200	56292	Mutillidae home page
/javascript/	301	169	JavaScript assets
/labs/	301	169	Labs directory
/passwords/	301	169	⚠Passwords directory — high interest, manually validated
/phpinfo.php	200	72694	⚠⚠Full phpinfo() output directly accessible — manually validated
/robots.txt	200	157	Robots exclusion file
/styles/	301	169	CSS styles directory
/webservices/	301	169	Web services directory

7.4 Manual Validation of Interesting Results

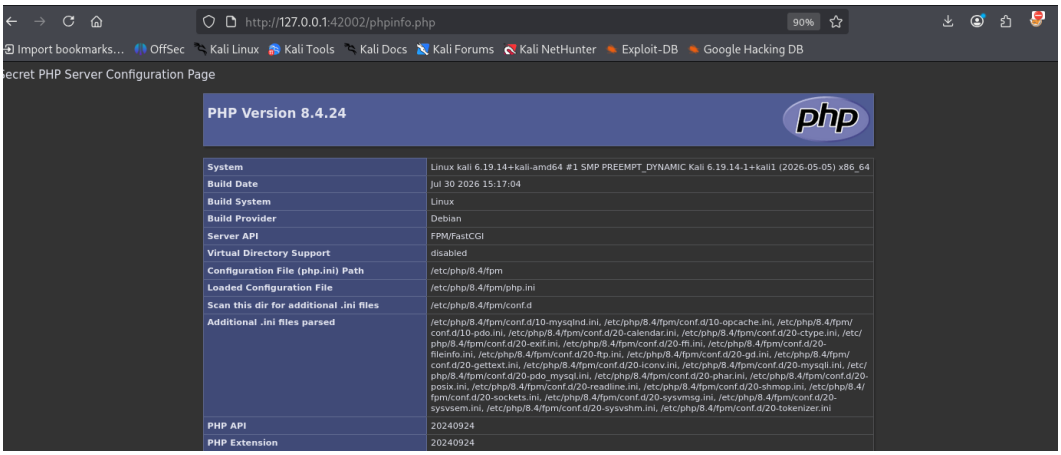
DVWA php.ini (200 OK, 154 bytes)

Manually visited http://127.0.0.1:42001/php.ini in browser. The file was directly served, revealing PHP configuration directives. This is an information-disclosure finding — php.ini should never be web-accessible as it may expose configuration details useful to an attacker for fingerprinting or exploitation planning.



Mutillidae phpinfo.php (200 OK, 72694 bytes)

Manually visited http://127.0.0.1:42002/phpinfo.php. A full phpinfo() output page was returned, exposing PHP version, all loaded extensions, server configuration, environment variables, and file system paths. This represents a significant information-disclosure finding in any real application.



Mutillidae .htaccess (200 OK, 690 bytes)

Manually visited http://127.0.0.1:42002/.htaccess. The file was directly served. An .htaccess file should never be returned to a client — it is an Apache server configuration file. Its exposure reveals server-side rewrite rules and access control directives, which is an information-disclosure and potential configuration-bypass finding.



8. Comparative Attack-Surface Diagram

The following diagram maps both applications side by side across the six key attack-surface categories required by the lab assessment: authentication, file upload, search, file inclusion, database, and session handling.

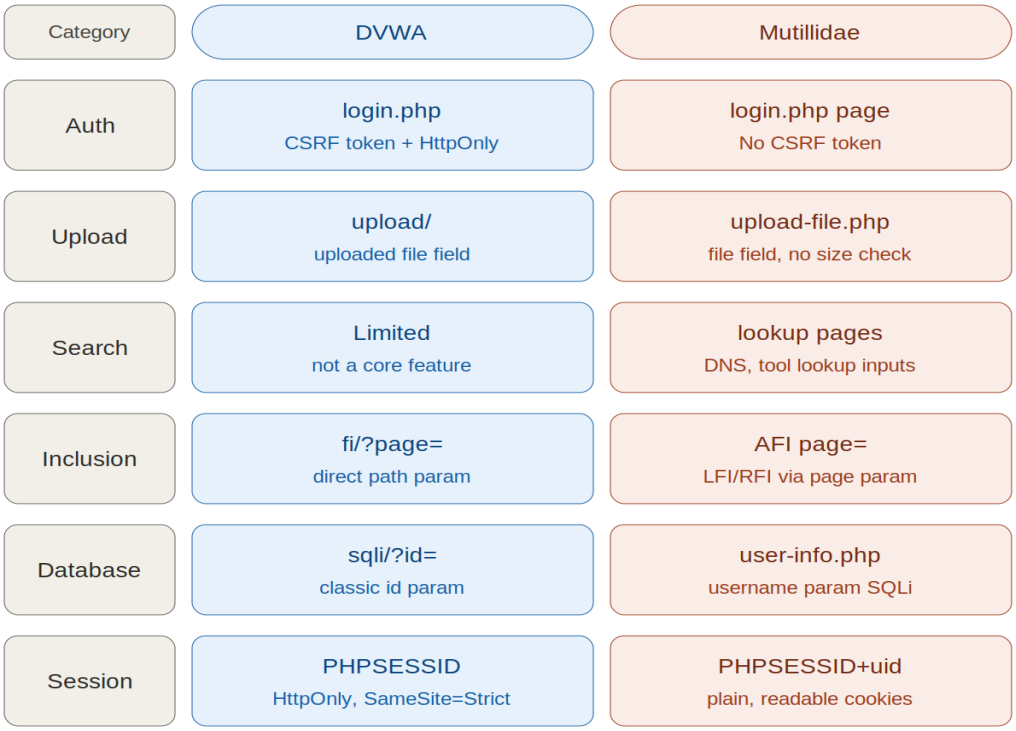


Figure: DVWA vs Mutillidae comparative attack-surface map across six vulnerability categories

Key observations from the comparison:

- Authentication: DVWA enforces a CSRF user_token on the login form and issues a rotated PHPSESSID with HttpOnly and SameSite=Strict attributes after successful authentication. Mutillidae has no CSRF token on its login form and stores identity in plain, script-accessible username and uid cookies after login.
- Session: DVWA's session cookie is protected against script access (HttpOnly) and cross-site submission (SameSite=Strict). Mutillidae's session management is significantly weaker — identity cookies are readable by any JavaScript on the page.
- Database: Both applications expose direct user-input-to-SQL-query flows. DVWA's canonical entry point is the id GET parameter in /vulnerabilities/sqli/. Mutillidae has multiple SQL injection surfaces across its OWASP-categorised modules (user-info.php, add-to-your-blog.php, etc.).
- File Inclusion: Both expose a page parameter that passes user-controlled filenames into server-side include operations — a classic LFI/RFI pattern in both /vulnerabilities/fi/ (DVWA) and arbitrary-file-inclusion.php (Mutillidae).
- Upload: Both expose unrestricted file upload endpoints with server-side validation as the only boundary. DVWA's upload path goes to /var/lib/dvwa/uploads/ (confirmed writable). Mutillidae's upload-file.php similarly accepts arbitrary files.
- Search: Mutillidae has substantially more search-like surfaces (DNS lookup, pen-test tool lookup, user info lookup) compared to DVWA, which does not have search as a dedicated vulnerability category. This gives Mutillidae a wider surface for injection testing via lookup-style inputs.

9. Analysis Questions

Q1. Why must deliberately vulnerable applications remain isolated?

DVWA and Mutillidae are intentionally built with unpatched, exploitable flaws — SQL injection, command injection, arbitrary file inclusion, and unrestricted file upload — with no hardening by design. If exposed on a routable network, they would allow any reachable attacker to fully compromise the host: read and write arbitrary files, execute operating system commands, pivot to other systems on the same network, or turn the machine into a platform for further attacks. There is no authentication or rate-limiting strong enough to compensate for a deliberately broken application stack.

In this lab, the risk was controlled using a defense-in-depth model. The VM sits behind a NAT boundary that VMware does not route to the physical LAN or internet — the 192.168.110.0/24 subnet exists only as a VMware-internal virtual network. Additionally, both applications' nginx configurations enforce allow 127.0.0.1; deny all;, rejecting any HTTP request not sourced from the VM itself. This means that even if an attacker somehow reached the VM's IP over the NAT link, nginx would block the request before it ever reached the PHP application code. Compromise of one layer does not expose the application.

Q2. Which inputs and trust boundaries appear most important for later testing?

The session and identity boundary is the most important finding from this lab. DVWA enforces a CSRF user_token on every state-changing form, issues a rotated PHPSESSID with HttpOnly and SameSite=Strict on login, and relies entirely on this server-side session for identity. Mutillidae, by contrast, issues no CSRF token, and stores identity post-login in plain, script-readable username and uid cookies (SameSite=Lax, not HttpOnly). This makes Mutillidae's session trust boundary significantly weaker and the most important area for later CSRF and session hijacking testing.

The direct parameter-to-backend flows are the second priority. Both applications pass raw user input from a URL or form parameter directly into sensitive operations: the id parameter into a SQL query in DVWA's /vulnerabilities/sqli/, the page parameter into a file-read operation in /vulnerabilities/fi/, and equivalent patterns in Mutillidae. These are the clearest injection and inclusion trust boundaries to target in later labs.

A notable cross-application boundary was also observed: since both applications run on 127.0.0.1 differing only by port, browser cookies are not port-scoped. Mutillidae's username and uid cookies were observed being transmitted in requests to DVWA. This is a lab-environment artifact rather than a vulnerability in either application, but it illustrates that origin-based trust assumptions do not always hold when multiple applications share a host.

Q3. Which automated discovery result required manual validation?

Four gobuster results required manual browser validation because automated status codes alone cannot confirm whether a result represents a genuine information-disclosure finding or a benign redirect/placeholder page.

The most significant was Mutillidae's phpinfo.php, which returned HTTP 200 with 72,694 bytes — a response large enough to suggest a full phpinfo() output. Manual validation confirmed this: the page exposed PHP version, all loaded extensions, server-side file paths, environment variables, and compile-time options. This is a high-value information-disclosure finding in any real application.

DVWA's php.ini (200 OK, 154 bytes) was the second finding requiring validation. A php.ini file being directly web-accessible is unexpected — manual confirmation showed the PHP configuration file was being served as plain text, revealing directive values.

Mutillidae's /passwords/ directory (301 redirect) and .htaccess file (200 OK, 690 bytes) were also manually validated. The .htaccess file being directly served is a misconfiguration finding — Apache configuration files should never be returned to a web client.

Q4. How do DVWA and Mutillidae differ in navigation, security modes and learning support?

Navigation: DVWA presents a flat left-hand menu of vulnerability categories with no further hierarchy. A tester can see every available module at a glance but has no organisational context for why those categories are grouped together. Mutillidae organises its content by OWASP Top 10 publication year (OWASP 2017, 2013, 2010, 2007) plus separate sections for Web Services, Others, and Labs. This means Mutillidae requires more navigation to find a specific vulnerability, but the organisation reflects how real penetration testing methodology is structured around OWASP classifications.

Security modes: DVWA uses a four-level dropdown (Low, Medium, High, Impossible) accessible via a dedicated DVWA Security page. Changing the level requires navigating to that page, selecting from the dropdown, and submitting — and the selected level is stored as a server-side session value, also reflected in a security cookie. Each level changes the actual PHP source code path used to handle requests. Mutillidae uses a binary Toggle Security switch in the top navigation bar, flipping instantly between insecure (Hosed) and secure modes without a page reload or separate navigation step. The mode is also exposed directly in the security cookie value (0 or 1), making it easier to manipulate from a proxy.

Learning support: Mutillidae is substantially more scaffolded for learners. It provides a Toggle Hints feature, a per-page "Help Me!" button, video tutorial links, a "What Should I Do?" guide, and a "Listing of Vulnerabilities" reference. DVWA provides only a static Instructions page per vulnerability module and relies on the user already understanding the vulnerability category. For self-directed learning, Mutillidae's guidance is considerably more accessible, though DVWA's simpler structure makes it easier to focus on a specific vulnerability without navigating a menu hierarchy.

10. Evidence Register

Ref	Application	Date/Time	Action	Output	Purpose
E1	Environment	26 Aug 2026 ~10:30	ip a, VMware adapter check, curl from Windows	ip_a_output.png, vmware_settings.png, 403 from Windows	Prove VM topology and isolation
E2	DVWA	26 Aug 2026 ~10:52	Login, set security=Low	dvwa_security_low.png	Confirm DVWA readiness and baseline security level
E3	DVWA	26 Aug 2026 ~10:55	Visit setup.php, click Create/Reset Database	dvwa_setup_check.png	DB reset proof and tech-fingerprint data
E4	Environment	26 Aug 2026	VMware Snapshot taken	Snapshot: clean-baseline-2026-08-26-dvwa-mutillidae	Clean recoverable baseline for both apps
E5	Mutillidae	26 Aug 2026 ~03:00	git clone, nginx vhost, DB user, set-up-database.php	mutillidae_home.png, mutillidae_reset_db.png	Mutillidae installation, DB rebuild and readiness proof
E6	Environment	26 Aug 2026 ~05:15	Burp proxy setup, about:config fix, FoxyProxy	firefox_proxy_settings.png	Proxy baseline configuration evidence
E7	DVWA	26 Aug 2026 ~05:13	Browse DVWA login, submit login POST	dvwa_burp_get_post.png	Required GET and POST capture for DVWA
E8	Mutillidae	26 Aug 2026 ~05:24	Browse Mutillidae login, submit login POST	mutillidae_burp_get_post.png (×2)	Required GET and POST capture for Mutillidae
E9	DVWA	26 Aug 2026 ~05:38	Submit dummy values to SQLi, FI, Upload, CMDi, XSS pages	dvwa_sqli.png, dvwa_fi.png, dvwa_upload.png	DVWA input point mapping
E10	Mutillidae	26 Aug 2026	Submit dummy values to SQLi, Upload, AFI pages	User-supplied screenshots	Mutillidae input point mapping

E11	Both	26 Aug 2026	gobuster with -t 2 --delay 300ms, manual browser validation	dvwa_gobuster.txt, mutillidae_gobuster.txt, validation screenshots	Low-rate content discovery and manual validation
E12	Both	26 Aug 2026	Comparative attack-surface diagram generated	attack_surface_comparison.png/.svg	Final required attack-surface diagram
E13	Both	26 Aug 2026	Analysis questions answered from lab observations	Section 9 of this report	Analysis and security reasoning

11. Activity Log

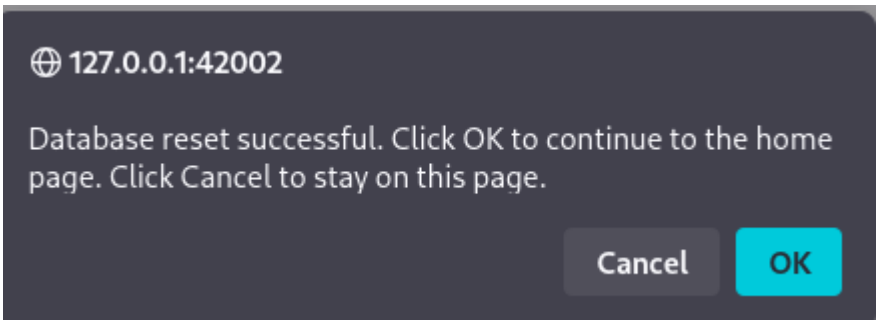
Time	Action	Tool	Error / Correction
~10:30	Confirmed VM NAT/Host-only adapter settings in VMware	VMware Workstation	None
~10:32	Ran ip a, confirmed eth0=192.168.110.141, eth1=no IP	Kali terminal	None
~10:35	Ran ss -tlnp, confirmed DVWA on 0.0.0.0:42001	Kali terminal	None
~10:36	Curl from Windows to 192.168.110.141:42001 → 403	Windows cmd	Initial curl used port 80 (wrong). Corrected to port 42001.
~10:40	Cat dvwa nginx vhost — found allow 127.0.0.1/deny all	Kali terminal	None — updated isolation evidence from residual-risk to defense-in-depth
~10:52	Logged into DVWA, set security to Low	Firefox / DVWA UI	Initial login silently failed in normal Firefox profile due to stale cookie. Resolved by clearing cookies for 127.0.0.1.
~10:55	Visited DVWA setup.php, clicked Create/Reset Database	Firefox / DVWA UI	None
~02:57	Attempted apt install mutillidae — not available	Kali terminal	Switched to git clone from official repo
~03:00	Cloned mutillidae repo, built nginx vhost on port 42002	Kali terminal	Multiple nginx config errors: relative snippet paths, port 80 conflict with Apache. Fixed by using absolute paths and a standalone nginx instance mirroring dvwa.service pattern.
~03:18	Created mutillidae.service systemd unit, started service	Kali terminal	None

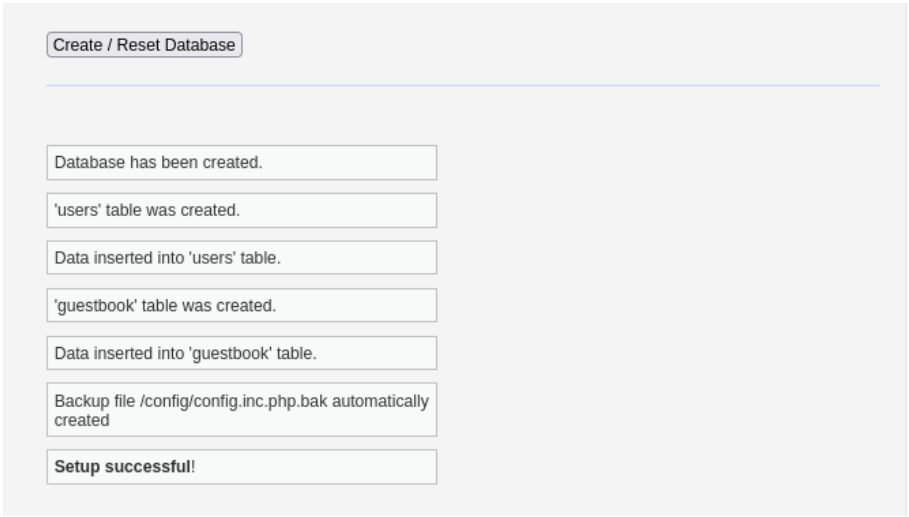
~03:21	Visited 127.0.0.1:42002 — DB offline error	Firefox	Schema tables missing. Clicked set-up-database.php link. Resolved.
~05:00	Configured Burp proxy, installed CA cert	Burp / Firefox	Firefox native proxy excludes 127.0.0.1 by default. Fixed via about:config network.proxy.allow_hijacking_localhost=true. Also: FoxyProxy disabled in private windows by default — enabled "Run in Private Windows" setting.
~05:13	Captured DVWA GET + POST login in Burp HTTP History	Burp Suite Community	None
~05:24	Captured Mutillidae GET + POST login in Burp HTTP History	Burp Suite Community	None
~05:38	Input point mapping — DVWA SQLi, FI, Upload, CMDi, XSS	Burp Intercept	Intercept was left ON; remembered to forward before continuing
~05:45	Input point mapping — Mutillidae SQLi, Upload, AFI	Burp Intercept	Screenshots captured by user directly
~06:00	Ran gobuster against both apps — low rate, in-scope only	gobuster	None
~06:10	Manual browser validation of 4 interesting discovery hits	Firefox	None
~06:20	Generated attack-surface comparison diagram	This session	None
~06:30	Drafted analysis question answers	This session	None

12. Cleanup and Restoration

The following cleanup steps were performed or are required before final submission:

- gobuster scans were stopped after completion and output saved to disk
- Burp proxy session was saved (or is available in Burp's project history)
- DVWA database reset via Setup/Reset DB page — baseline state confirmed
- Mutillidae database reset via Reset DB link — baseline state confirmed
- Clean snapshot (clean-baseline-2026-08-26-dvwa-mutillidae) verified available in VMware Snapshot Manager
- Neither DVWA (port 42001) nor Mutillidae (port 42002) became reachable from an uncontrolled network — nginx allow/deny ACL remained active throughout, confirmed by repeated 403 responses from Windows host during testing





Out-of-scope confirmation: No system other than http://127.0.0.1:42001/ and http://127.0.0.1:42002/ was accessed or tested during this lab. All tool output, screenshots and captures in this report were generated solely against the locally installed DVWA and OWASP Mutillidae II instances inside the isolated VM.